TETRIS 67

Rapport de Conception Technique

Clone Tetris moderne · Rendu DOM/CSS 3D pur · Esthétique Nintendo 64 (1996)

Moteur de jeu Pure JavaScript, zéro dépendance	**Rendu visuel** DOM/CSS 3D - zéro canvas, zéro WebGL
Standard SRS officiel, 7-bag, scoring complet	**Objectif** 60 FPS - desktop & mobile

Présenté par :

- Bilal Bachar
- Naïm Harib
- Salima Mazaev

Table des matières

1. Présentation et Architecture Globale 3
1.1 Objectifs du projet 3
1.2 Vue d'ensemble architecturale 3
1.3 Boucle principale (main.js) 3
2. Moteur de Jeu Pur (src/core) 4
2.1 Modèle de données 4
2.2 Les 7 tétrominos (pieces.js) 4
2.3 7-Bag Randomizer (bag.js) 4
2.4 Super Rotation System (srs.js) 4
2.5 Détection T-Spin (tspin.js) 5
2.6 Scoring (scoring.js) 5
2.7 Orchestrateur game.js 5
3. Rendu 3D CSS et Système de Caméra 6
3.1 Parti pris : DOM/CSS 3D 6
3.2 Chaîne de transformation 3D 6
3.3 Caméra virtuelle (camera.js + math3d.js) 6
3.4 Effets de caméra (effects.js) 7
3.5 Caméra troisième personne du Hub 7
3.6 FX volatils 7
4. Scènes et Flux de Navigation 8
4.1 sceneManager 8
4.2 Transitions (scenes/transitions.js) 8
4.3 Écran titre (titleScene.js) 8
4.4 Hub (hubScene.js) 8
4.5 Game (gameScene.js) 9
4.6 Schéma de navigation 9
5. Inputs, Audio et Persistance 10
5.1 Abstraction d'inputs (input/actionMap.js) 10
5.2 Clavier avec DAS/ARR (input/keyboard.js) 10
5.3 Tactile (input/touch.js) 10
5.4 Audio (audio/soundManager.js) 10
5.5 Persistance (ui/storage.js) 11
6. Qualité Logicielle, Tests et Performances 12
6.1 Tests unitaires (tests/) 12
6.2 Performances 12
6.3 Accessibilité et Responsive 12
6.4 Points forts de la conception 13
6.5 Limitations connues et améliorations possibles 13
6.6 Conclusion 13

1. Présentation et Architecture Globale

1.1 Objectifs du projet

TETRIS 67 est un clone moderne respectant rigoureusement les directives officielles Tetris, SRS, 7-bag, scoring complet, enveloppé dans une esthétique rappelant les jeux Nintendo 64 de 1996.

Le parti pris technique majeur est une contrainte auto-imposée :

- Zéro canvas, zéro WebGL
- Rendu 3D intégral via des transformations DOM/CSS 3D pures
- 60 FPS visés sur navigateurs desktop et mobile
- Aucune dépendance externe à l'exécution

1.2 Vue d'ensemble architecturale

L'architecture s'articule autour de cinq couches strictement séparées :

Couche	Rôle
CORE	Logique métier pure, testable en Node, totalement indépendante du DOM
RENDER	Lit l'état du core pour mettre à jour l'arbre DOM sans jamais muter la logique
SCENES	Orchestre l'assemblage entre core, renderers et systèmes transverses
SYSTEMS	Gère les I/O et effets globaux (caméra, audio, input, effects)
UI	Superpose les interfaces 2D (écrans, popups d'événements)

La dépendance est unilatérale et descendante : UI → SCENES → RENDER → CORE, avec SYSTEMS en accès transverse depuis SCENES.

1.3 Boucle principale (main.js)

La synchronisation de l'état global s'effectue via requestAnimationFrame selon un ordre de mise à jour critique à chaque frame :

```
keyboard/touch → effects → sceneManager → camera → skybox → fog → particles
```

Point critique : les effets (secousses, inclinaisons) doivent être évalués avant sceneManager afin que la caméra puisse composer sa transformation de base avec les offsets transitoires générés par les effets visuels.

2. Moteur de Jeu Pur (src/core)

2.1 Modèle de données

Grille	22×10 cellules - 2 lignes de débordement cachées + 20 lignes visibles. Tableau 2D immuable ; chaque cellule contient 0 (vide) ou l'identifiant de pièce (1 à 7). Toute opération retourne une nouvelle instance.

Pièce active	Objet ActivePiece { type, x, y, rotation, id } - coordonnées pointant le coin supérieur gauche de la matrice englobante.

2.2 Les 7 tétrominos (pieces.js)

Les pièces sont définies via des matrices binaires, déclinées pour les 4 états de rotation (0, R, 2, L) :

- **Matrices 3×3 :** J, L, S, T, Z
- **Matrices 4×4 :** I, O

Ces matrices sont converties à l'initialisation en listes de coordonnées {x, y} relatives. L'ensemble est soumis à un Object.freeze profond pour garantir l'immutabilité et optimiser la compilation JIT (V8).

2.3 7-Bag Randomizer (bag.js)

La génération de pièces suit la norme 7-bag officielle :

- Un sac contient une instance de chaque tétromino
- Mélange par algorithme Fisher-Yates à chaque épuisement du sac
- **peek(n)** et **clone()** : alimentent la Next Queue sans altérer le générateur
- **PRNG injectable** (createSeededRng / mulberry32) : garantit la reproductibilité lors des tests unitaires

2.4 Super Rotation System (srs.js)

L'implémentation SRS gère collisions et décalages muraux (wall kicks) avec des tables d'offsets spécifiques à chaque pièce :

- Table standard - pièces J, L, S, T, Z
- Table spécifique - pièce I
- Table symétrique additionnelle - rotations à 180°

La fonction **tryRotate()** tente d'abord la rotation naïve (kick 0), puis itère sur les 4 kicks de secours. Elle retourne un RotationResult structuré **{ success, kickIndex, offset }** requis par la détection T-Spin.

2.5 Détection T-Spin (tspin.js)

Règle	Condition
Condition de base	Dernière action = rotation de la pièce T ET au moins 3 des 4 coins de sa matrice 3×3 occupés
Statut Proper	Les deux coins avant sont remplis
Statut Mini	Appliqué si les deux coins avant ne sont pas tous deux remplis
Exception TST kick	Un Mini est élevé en Proper si le kickIndex 4 est utilisé

2.6 Scoring (scoring.js)

Le calcul des points suit strictement les guidelines officielles Tetris :

Type	Formule
Lignes	base_points(clearId) × level × B2B_multiplier (×1.5 si Back-to-Back éligible)
Combos	50 × combo × level
Soft Drop	1 point par cellule parcourue
Hard Drop	2 points par cellule parcourue
Perfect Clear	Bonus massif variant selon le type de ligne effacée

Formule de gravité : `(0.8 - ((level - 1) × 0.007))^(level - 1)`

2.7 Orchestrateur game.js

Le moteur central opère comme une machine à états finis :

READY → SPAWNING → PLAYING → LOCKING → CLEARING → GAME_OVER

- **LOCKING :** délai de 500 ms avec maximum 15 réinitialisations (anti-stall)
- **EventEmitter pur :** diffuse les événements métier (SPAWN, LOCK, LINES_CLEARED, TSPIN, PERFECT_CLEAR) traduits en stimuli visuels/sonores par la scène

3. Rendu 3D CSS et Système de Caméra

3.1 Parti pris : DOM/CSS 3D

Le choix du DOM 3D offre des avantages architecturaux significatifs :

- Héritage natif des variables CSS (design tokens)
- Inspection hiérarchique via DevTools navigateur
- Utilisation du moteur de composition GPU natif pour les transitions

La limitation inhérente (absence de shaders programmables) est palliée par une utilisation intensive de dégradés radiaux et linéaires pour simuler l'éclairage ambiant.

3.2 Chaîne de transformation 3D

Perspective	#viewport - perspective: 1200px
World	#world (transform-style: preserve-3d) - reçoit la matrice inverse de la caméra
Cube	6 faces DOM positionnées par rotateX/Y/Z + translateZ avec backface-visibility: hidden
Éclairage	Cuit dans les CSS : face supérieure = plus claire, avant = moyenne, autres assombries par paliers

3.3 Caméra virtuelle (camera.js + math3d.js)

La caméra ne bouge pas l'écran, elle déplace l'univers. Une translation +Z caméra correspond à un translateZ(−z) sur le conteneur #world.

Méthode	Description
set(preset)	Applique un preset de caméra prédéfini
moveTo(target, ms, easing)	Animation interpolée vers une cible
update(dt)	Mise à jour par delta-time à chaque frame

Les angles sont interpolés via des quaternions ou des lerps circulaires pour éviter les sauts de cardan (gimbal lock) lors des rotations de ±180°.

3.4 Effets de caméra (effects.js)

Effet	Description
Shake	Algorithme de bruit (noise1) sur X/Y avec enveloppe de décroissance quadratique, 4 intensités préconfigurées
Punch	Impulsion Z/rotX très brève (fonction sinus) réagissant au Hard Drop
Tilt	Décalage rotatif Z persistant signifiant accumulation de combos ou zone de danger
Flash	Div plein-écran avec mix-blend-mode modifiant l'exposition globale

3.5 Caméra troisième personne du Hub

Un contrôleur spécialisé calcule la position via coordonnées polaires ancrées sur la mascotte, avec poursuite lissée :

- **Position :** lerp de 0.12
- **Rotation Y :** lerpAngle de 0.10

La caméra est volontairement rigide (non pilotable à la souris) pour respecter le gameplay contraint de l'époque N64 et simplifier les interactions DOM.

3.6 FX volatils

Effet	Implémentation
Skybox	Composée par box-shadow multiple pour les étoiles, liée uniquement à l'axe rotY de la caméra
Fog	Calque en dégradé radial, opacité et couleur (var(--fog-color)) réagissent à l'état du jeu
Particles	Pool de 100 éléments DOM recyclés, évite le Garbage Collection
textPop	Composant UI projeté en 2D via coordonnées écran, animations CSS keyframées

4. Scènes et Flux de Navigation

4.1 sceneManager

Le sceneManager gère un registre de factories de scènes. Le cycle de vie de chaque scène respecte le contrat suivant :

`mount → update → onPause → onResume → onResize → destroy`

Un objet Context global est injecté dans chaque scène, mutualisant l'accès à la caméra, aux entrées, à l'audio et au bus d'événements. Les transitions entre scènes sont asynchrones, la nouvelle scène ne monte qu'après la fin de l'animation OUT.

4.2 Transitions (scenes/transitions.js)

Le projet intègre une transition signature de type line-clear Tetris, en deux phases :

- **Phase OUT :** Génération d'une grille DOM 2D en surimpression. Les blocs chutent avec staggering ligne par ligne.
- **Phase IN :** Une fois l'écran rempli et la nouvelle scène chargée, les blocs clignotent, rétrécissent vers leur centre, puis disparaissent du bas vers le haut.

4.3 Écran titre (titleScene.js)

Composé de deux entités DOM principales :

- **Logotype "TETRIS" :** effet métal biseauté, contour néon
- **"67" :** doré incandescent

Neuf cubes 3D orbitent perpétuellement autour du centre. Les trajectoires sont précalculées en coordonnées polaires dans le constructeur pour éviter tout saut visuel au chargement (FOUC 3D).

4.4 Hub (hubScene.js)

Environnement 3D généré à partir d'un manifeste JSON (/public/maps/hub-default.json). Les murs sont construits dynamiquement depuis des segments 2D projetés en CSS 3D.

Le joueur contrôle un avatar cubique interactif avec simulation physique basique pour 4 mèches (inertie relative à l'accélération). La collision avec les tableaux interactifs active un halo CSS et débloque l'action INTERACT pour basculer vers gameScene.

4.5 Game (gameScene.js)

Le contrôleur principal lie l'état abstrait (game.js) à la présentation (boardRenderer.js). Les événements du moteur sont traduits en feedback sensoriel immédiat :

LINES_CLEARED	Effacement DOM du Board + effects.applyShake() + 10 particules shard par ligne + AudioBuffer correspondant

4.6 Schéma de navigation

Depuis	Vers
[Démarrage]	→ titleScene
titleScene	→ hubScene (Transition Tetris)
hubScene	→ gameScene (Interaction Tableau)
gameScene	→ ui/screens (Épuisement grille ou objectif atteint)
ui/screens	→ gameScene (Rejouer) · hubScene (Hub) · titleScene (Quitter)

5. Inputs, Audio et Persistance

5.1 Abstraction d'inputs (input/actionMap.js)

Les contrôles matériels publient des identifiants sémantiques, découplant totalement la logique de présentation du matériel de saisie :

- MOVE_LEFT, MOVE_RIGHT, ROTATE_CW, ROTATE_CCW
- HARD_DROP, SOFT_DROP, HOLD

L'actionMap maintient une pile de contextes (ex: game vs menu). Seul le contexte au sommet résout et propage les actions, empêchant les conflits entre scènes.

5.2 Clavier avec DAS/ARR (input/keyboard.js)

Fonctionnalité	Détail
DAS (Delayed Auto Shift)	170 ms - délai avant déclenchement de la répétition
ARR (Auto Repeat Rate)	50 ms par déplacement après expiration du DAS
Last Key Wins	Annule le DAS de la flèche gauche si la droite est pressée simultanément
Ghost keys	Écouteur sur blur de la fenêtre → releaseAll forcé

5.3 Tactile (input/touch.js)

Geste	Action
Tap court	ROTATE_CW
Swipe horizontal	Quantifié par pas de 32 px → actions MOVE sans DAS
Swipe bas court	SOFT_DROP
Swipe bas rapide	HARD_DROP (vélocité > seuil)
Appui prolongé	HOLD

5.4 Audio (audio/soundManager.js)

L'architecture audio combine deux technologies complémentaires :

- **Web Audio API :** décode et gère les SFX via un pool de BufferSourceNode, latence quasi-nulle indispensable au gameplay Tetris
- **HTMLAudioElement :** streaming de la musique de fond avec fondus croisés manuels

Un mécanisme d'auto-unlock écoute le premier événement d'interaction global pour débloquer le contexte audio (AudioContext.resume()), respectant les politiques de lecture automatique des navigateurs.

5.5 Persistance (ui/storage.js)

Wrapper défensif autour de window.localStorage :

- Chaque appel protégé par try/catch (quotas, navigation privée Safari)
- Données sérialisées en JSON sous des clés scopées

Exemples de clés	tetris64.scores.marathon · tetris64.settings.volume

6. Qualité Logicielle, Tests et Performances

6.1 Tests unitaires (tests/)

Le framework Vitest assure la validation du core engine sans nécessiter de mock DOM, grâce à l'architecture découplée :

Fichier	Couverture
bag.test.js	Vérifie la complétude des cycles de 7 pièces et la reproductibilité via RNG seedé
srs.test.js	Valide les matrices de rotation cycliques et le fallback sur les 5 offset kicks
tspin.test.js	Teste les 4 orientations du T dans des encoches serrées, coins occupés, promotion Mini→Proper
scoring.test.js	Garantit la précision mathématique des multiplicateurs B2B, combos et Perfect Clear

6.2 Performances

Technique	Description
DOM Diffing manuel	La grille n'est pas recréée à chaque frame. Un miroir Map("y,x" → DOMElement) est maintenu, seules les cellules modifiées reçoivent une nouvelle classe CSS
Accélération matérielle	will-change: transform sur les entités mobiles → création de calques GPU dédiés
Budget particules	Figé à 100 éléments maximum - nœuds inactifs en display: none plutôt que retirés du DOM
Pas de temps plafonné	dt limité à 100 ms pour éviter la spirale de la mort au retour depuis un onglet inactif

6.3 Accessibilité et Responsive

- **prefers-reduced-motion :** durées d'animation écrasées à 0/200 ms, shake neutralisé, skybox stoppée
- **Responsive :** sous 768 px, --cube-size et la perspective globale sont proportionnellement réduites
- **Navigation clavier :** Tab, Enter, Escape - attributs ARIA sur les contrôles globaux (mute)

6.4 Points forts de la conception

L'accomplissement technique principal réside dans la séparation absolue entre le cerveau mathématique (core) et la marionnette visuelle (DOM). La dépendance unilatérale vers un bus d'événements permet au moteur de tourner à 1000 FPS en backend de test, tandis que le frontend se contente d'écouter les signaux pour animer le CSS.

Le format de carte JSON pour le hub permet aux designers de modifier le niveau sans intervention dans le code source.

6.5 Limitations connues et améliorations possibles

- Physique du Hub sommaire : clamp rectangulaire au lieu d'une vraie détection de collision par lancer de rayon contre les murs 3D
- Les dégradés CSS directionnels n'offrent pas d'ombres portées dynamiques entre objets 3D
- Architecture mono-thread : calculs mathématiques liés au reflow du navigateur, sensible aux variations lourdes de l'arbre DOM sur appareils bas de gamme
- Absence de WebSocket : pas de multijoueur synchrone ni d'envoi de garbage lines

6.6 Conclusion

> *TETRIS 67 démontre la viabilité du DOM web standard comme moteur de jeu 3D performant. En couplant un moteur mathématique pur, testable à 100 %, avec l'API CSS 3D moderne, l'architecture délivre une expérience de jeu fluide, respectueuse des normes esportives Tetris, tout en affichant une identité visuelle rétro-1996 distinctive, sans nécessiter d'infrastructures Canvas ou WebGL complexes.*